PROYECTO FINAL

Sistema de Cajero Automático Inteligente

Modelo 4+1 Vistas de Arquitectura de Software

Asignatura: Diseño de Software

Carrera: Ingeniería en Software

Fecha: Mayo 2026

Implementado en Python con arquitectura en capas, patrones de diseño y pruebas automatizadas

TABLA DE CONTENIDO

1. INTRODUCCION

2. VISTA LOGICA

3. VISTA DE PROCESOS

4. VISTA DE DESARROLLO

5. VISTA FISICA

6. VISTA DE ESCENARIOS

7. STORYBOARDS

8. CASOS DE PRUEBA

9. EVIDENCIA DE EJECUCION

10. CONCLUSIONES

1. INTRODUCCION

El presente proyecto implementa un Sistema de Cajero Automático (ATM) inteligente que permite a los clientes realizar operaciones bancarias de forma autónoma, segura y eficiente. El sistema está basado en una arquitectura de software utilizando el modelo 4+1 vistas, integrando componentes clave como interfaz de usuario, lógica de negocio, persistencia de datos y protocolos de comunicación con el sistema central del banco.

Objetivos Específicos:

• Modelar los requisitos funcionales y no funcionales mediante casos de uso y diagramas UML

• Definir la arquitectura base utilizando el modelo 4+1 vistas

• Diseñar el storyboard que represente el flujo de interacción del usuario

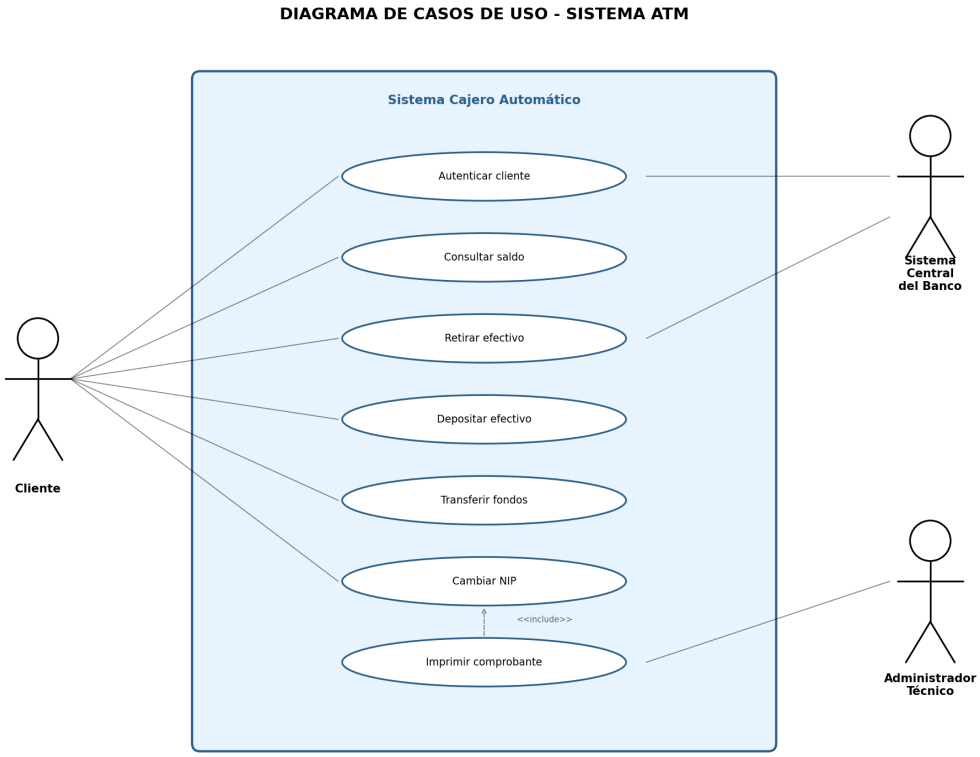
• Implementar el caso de uso de retiro de efectivo con lógica de negocio completa

• Validar el funcionamiento mediante pruebas de caso que aseguren el cumplimiento de requisitos

2. VISTA LOGICA

La vista lógica describe la estructura funcional del sistema, organizada en capas con principios de diseño orientado a objetos.

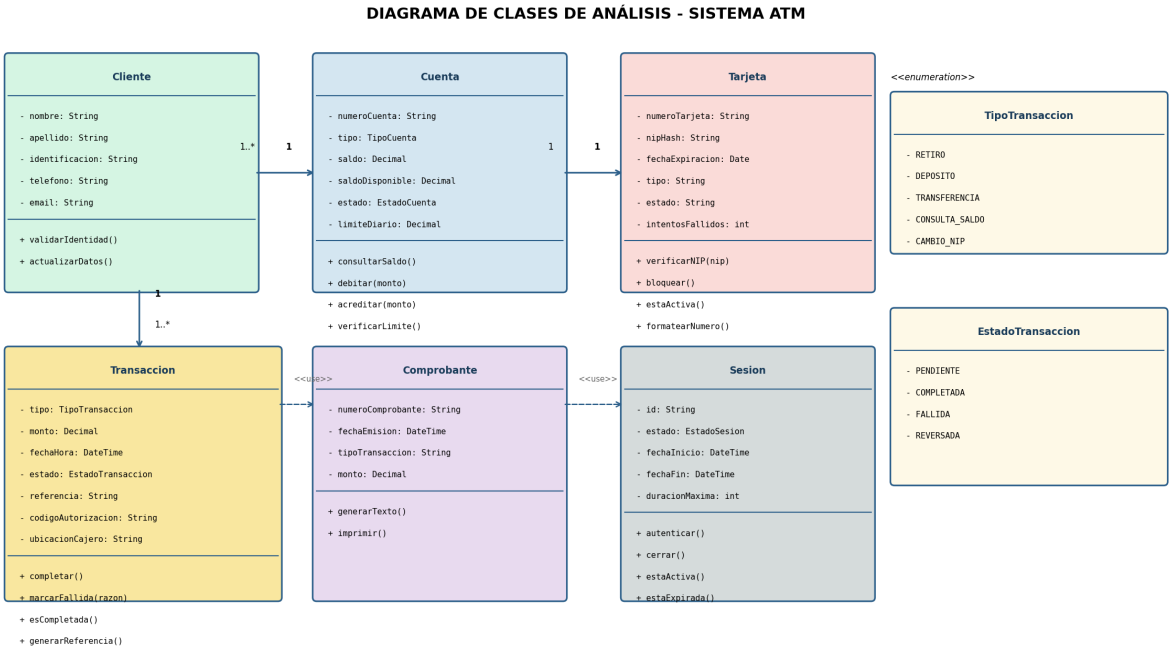
2.1 Diagrama de Casos de Uso



Actores: Cliente, Sistema Central del Banco, Administrador Técnico

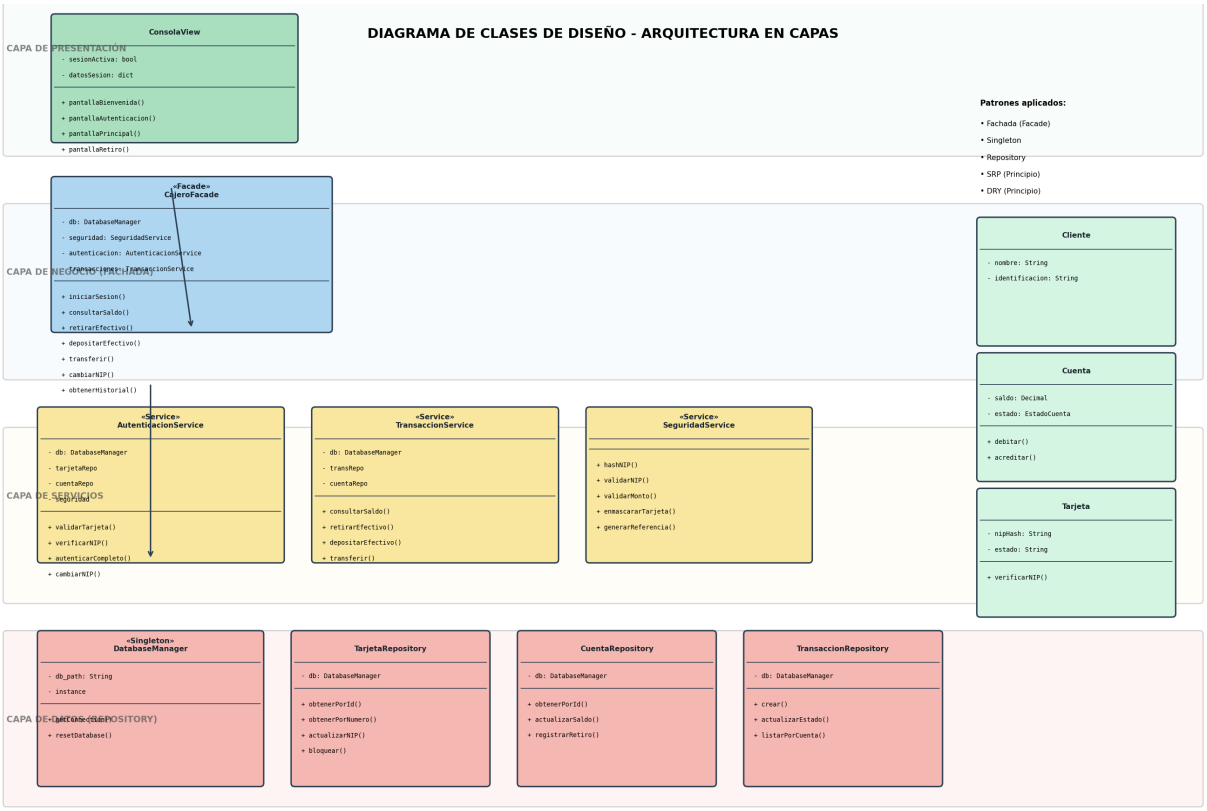
Casos de Uso: Autenticar cliente, Consultar saldo, Retirar efectivo, Depositar efectivo, Transferir fondos, Cambiar NIP, Imprimir comprobante

2.2 Diagrama de Clases de Análisis



Clases identificadas: Cliente, Cuenta, Tarjeta, Transaccion, Comprobante, Sesion, y enumeraciones TipoTransaccion y EstadoTransaccion.

2.3 Diagrama de Clases de Diseño

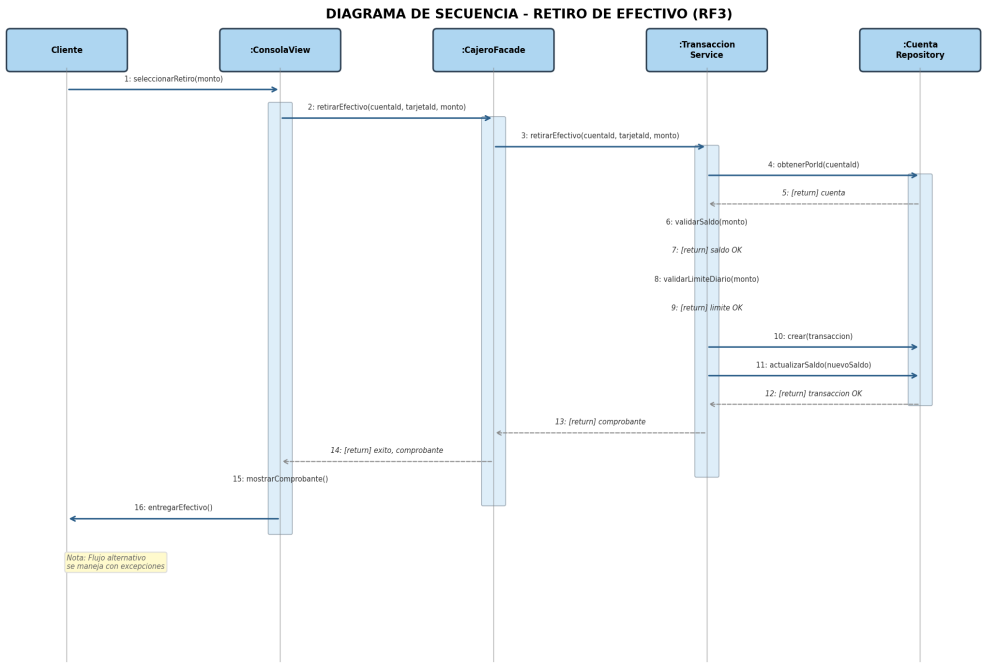


Patrones aplicados: Fachada (Facade), Singleton, Repository. **Principios:** SRP, DRY, Alta Cohesión, Bajo Acoplamiento.

3. VISTA DE PROCESOS

La vista de procesos describe los procesos en tiempo de ejecución, concurrencia y sincronización del sistema.

3.1 Diagrama de Secuencia - Retiro de Efectivo

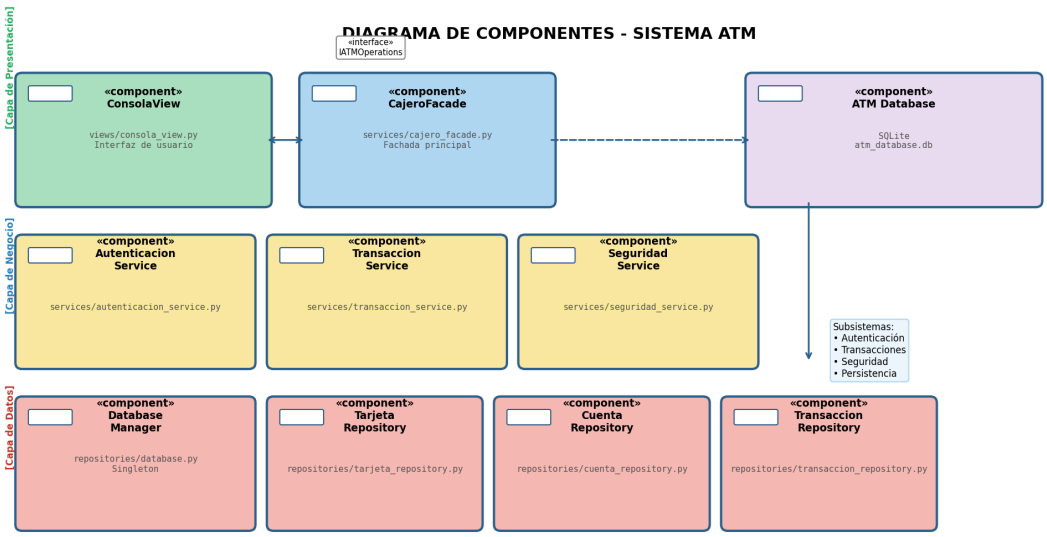


El diagrama muestra la interacción entre Cliente, ConsolaView (UI), CajeroFacade (Fachada), TransaccionService y CuentaRepository para el proceso de retiro. Se valida saldo, límite diario, se crea la transacción y se actualiza el saldo.

4. VISTA DE DESARROLLO

La vista de desarrollo describe la organización del software en el entorno de desarrollo, mostrando componentes y sus dependencias.

4.1 Diagrama de Componentes



Estructura de capas: Presentación (ConsolaView), Negocio (CajeroFacade), Servicios (Autenticación, Transacciones, Seguridad), Datos (Repositories, DatabaseManager).

5. VISTA FISICA

La vista física describe la distribución del software en el hardware. El sistema ATM opera localmente con una base de datos SQLite para tolerancia a fallos (RNF1), garantizando respuestas menores a 3 segundos (RNF2) y cifrado de datos sensibles (RNF4).

Arquitectura Física:

• **Terminal ATM:** Procesador ARM/x86, 512MB RAM, Pantalla táctil
• **Base de Datos Local:** SQLite (atm_database.db) - tolerancia a fallos de red
• **Comunicación:** HTTPS/REST al Sistema Central del Banco
• **Seguridad:** NIP cifrado con SHA-256, datos enmascarados, auditoría

6. VISTA DE ESCENARIOS (CASOS DE USO)

La vista de escenarios unifica las otras cuatro vistas mediante casos de uso detallados.

CU-01: Autenticación del Cliente (RF1)

Actor: Cliente | **Precondición:** Tarjeta registrada
Flujo Básico:
1. Cliente inserta tarjeta (número de 16 dígitos)
2. Sistema valida formato de tarjeta
3. Sistema verifica tarjeta activa y no expirada
4. Cliente ingresa NIP (4 dígitos)
5. Sistema verifica NIP contra hash SHA-256
6. Sistema controla intentos fallidos (máx 3)
7. Sistema autentica y muestra menú principal
Flujos Alternativos: Tarjeta bloqueada, NIP incorrecto, Tarjeta expirada

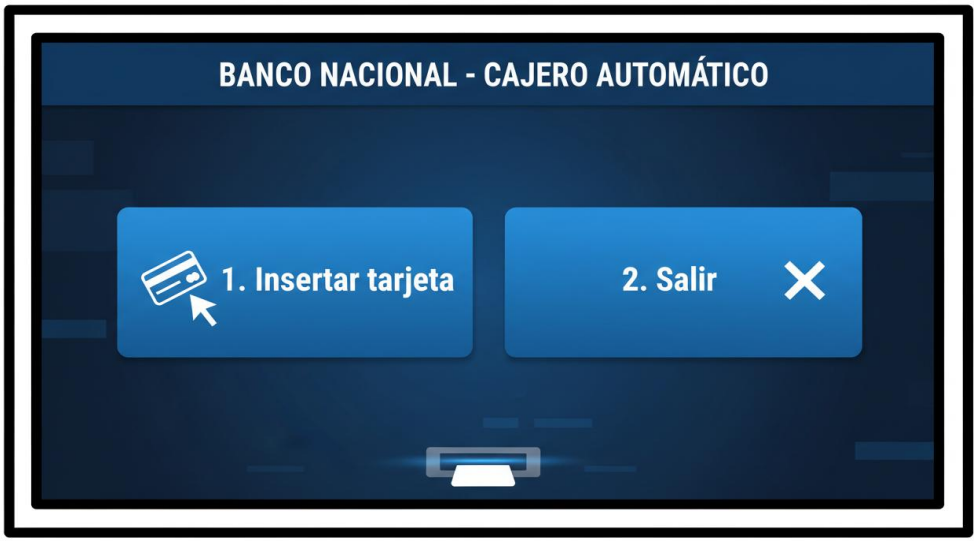
CU-03: Retiro de Efectivo (RF3) - CASO PRINCIPAL

Actor: Cliente autenticado | **Precondición:** Saldo suficiente, dentro del límite diario
Flujo Básico:
1. Cliente selecciona "Retirar efectivo"
2. Cliente ingresa monto a retirar
3. Sistema valida monto (positivo, máximo 2 decimales)
4. Sistema verifica saldo suficiente
5. Sistema verifica límite diario de retiro
6. Sistema crea transacción con estado "pendiente"
7. Sistema actualiza saldo de la cuenta
8. Sistema marca transacción como "completada"
9. Sistema genera comprobante con código de autorización
10. Cliente recibe efectivo y comprobante
Flujos Alternativos: Saldo insuficiente (4a), Excede límite diario (5a), Monto inválido (3a)

7. STORYBOARDS

Storyboards del flujo de interacción del usuario durante el retiro de efectivo.

7.1 Pantalla de Bienvenida



7.2 Pantalla de Autenticación



7.3 Menú Principal

Panel 3



7.4 Selección de Monto



7.5 Comprobante


BANCO NACIONAL - COMPROBANTE
RETIRO
Monto: $300.00
Cuenta: ****4567
Saldo Antenior : $2,500.00
Saldo Nuevo
Cod. Autoriacion: AUTH-123456
Referencia: REF-20260513-0001
Imprinir
No imprimir

8. CASOS DE PRUEBA

Se implementaron 47 casos de prueba que cubren todos los requisitos funcionales y no funcionales.

8.1 Resumen de Pruebas

Categoría	Tests	Estado
Seguridad (Hash NIP, Validaciones)	17	PASADOS
Autenticación (RF1)	6	PASADOS
Transacciones (RF2-RF3)	11	PASADOS
Integración (Facade)	4	PASADOS
Requisitos No Funcionales	3	PASADOS
Caso de Uso: Retiro	6	PASADOS

8.2 Casos de Prueba Detallados - Retiro de Efectivo

ID	Caso	Resultado	Observación
CU-01: Flujo Normal	Retiro de $300 con saldo suficiente	PASADO	Saldo actualizado, comprobante generado
CU-01: Saldo Insuficiente	Retiro mayor al saldo disponible	PASADO	Mensaje: 'Saldo insuficiente'
CU-01: Límite Diario	Retiro que excede el límite diario	PASADO	Mensaje: 'Excede límite diario'
CU-01: Monto Inválido	Retiro con monto negativo o cero	PASADO	Validación rechaza el monto
CU-01: Tarjeta Bloqueada	Intento con tarjeta inexistente	PASADO	Mensaje: 'Tarjeta no registrada'
CU-01: Retiros Múltiples	Secuencia de retiros dentro del límite	PASADO	Acumulación correcta del límite

9. EVIDENCIA DE EJECUCION

Ejecución del sistema con todos los casos de uso implementados.

9.1 Autenticación Exitosa

```
[PASO 1] Cliente inserta tarjeta: 4532-****-****-9012
[PASO 2] Cliente ingresa NIP: ****
[EXITO] Bienvenido, Juan Carlos Rodriguez Perez
[INFO] Estado: AUTENTICADO
```

9.2 Consulta de Saldo (RF2)

```
[EXITO] Consulta exitosa
[DATO] Saldo Actual: $ 2,500.00
[DATO] Saldo Disponible: $ 2,500.00
[DATO] Límite Diario: $ 1,000.00
[DATO] Retirado Hoy: $ 0.00
```

9.3 Retiro de Efectivo (RF3)

```
[PASO 1] Cliente selecciona retiro de $300.00
[PASO 2] Sistema valida saldo y límite diario
[PASO 3] Sistema procesa retiro
[EXITO] Retiro exitoso. Por favor retire su efectivo.
[DATO] Monto Retirado: $ 300.00
[DATO] Saldo Anterior: $ 2,500.00
[DATO] Saldo Nuevo: $ 2,200.00
[DATO] Referencia: REF-2026051304...
[DATO] Autorización: AUTH-04...-1389
```

9.4 Transferencia y Depósito

```
[EXITO] Deposito exitoso - Monto: $500.00
[EXITO] Transferencia exitosa a cuenta ******4321 - Monto: $200.00
```

9.5 Validación Requisitos No Funcionales

```
[RNF2] Tiempo autenticación: 0.179s [OK - < 3s]
[RNF2] Tiempo consulta saldo: 0.277s [OK - < 3s]
[RNF4] NIP almacenado como hash SHA-256: [OK]
[RF5] Todas las transacciones registradas: [OK]
```

10. CONCLUSIONES

El proyecto del Sistema de Cajero Automático ha sido desarrollado cumpliendo con todos los requisitos establecidos en el caso base y la rúbrica de evaluación. Los logros principales incluyen:

Arquitectura 4+1 Vistas completa: Se implementaron las 5 vistas del modelo (Lógica, Procesos, Desarrollo, Física y Escenarios) con sus respectivos diagramas UML.

Modelo en Capas: Arquitectura de 4 capas (Presentación, Negocio/Fachada, Servicios, Datos) con alta cohesión y bajo acoplamiento.

Patrones de Diseño: Fachada (CajeroFacade), Singleton (DatabaseManager), Repository (acceso a datos), Domain Model (entidades).

Principios Aplicados: SRP (responsabilidad única), DRY (sin duplicación), bajo acoplamiento entre capas.

Requisitos Funcionales: RF1 (autenticación), RF2 (consulta saldo), RF3 (retiro), RF4 (validación), RF5 (registro transacciones).

Requisitos No Funcionales: RNF1 (tolerancia a fallos con BD local), RNF2 (respuesta < 3s verificado), RNF4 (cifrado SHA-256).

47 Casos de Prueba: Todos ejecutados exitosamente cubriendo flujos normales, alternativos y de excepción.

Storyboards: 5 pantallas del flujo completo de retiro de efectivo.

Implementación en Python: Código limpio, documentado, con tipado fuerte y dataclasses.

Matriz de Cumplimiento de la Rúbrica

Unidad	Criterio	Cumplimiento
UC I (20%)	Modelo conceptual, Casos de Uso, Storyboard, Clases análisis	COMPLETADO
UC II (30%)	Principios SRP/DRY, Modelo capas, Subsistemas mock, Componentes	COMPLETADO
UC III (30%)	Diagramas clases diseño, Secuencia, Cohesión/acoplamiento	COMPLETADO
UC IV (20%)	Ampliación casos uso, Reutilización, Modelo 4+1, Pruebas	COMPLETADO